Alexandria Engineering Journal 111 (2025) 530–539

Contents lists available at ScienceDirect

Alexandria Engineering Journal

journal homepage: www.elsevier.com/locate/aej

Original article

3D-AOCL: Analytic online continual learning for imbalanced 3D point cloud classification

Ziqian Zeng [a], Jianwei Wang [a], Lin Wu [b], Weikai Lu [a], Huiping Zhuang [a,*]

[a] Shien-Ming Wu School of Intelligent Engineering, South China University of Technology, Guangzhou, 510641, China
[b] School of Future Technology, South China University of Technology, Guangzhou, 510641, China

ARTICLE INFO

Keywords:
Point cloud classification
Autonomous driving
Online continual learning
Analytic learning

ABSTRACT

Recent autonomous driving systems heavily rely on 3D point cloud data collected from multiple sensors for environmental awareness and decision-making. However, it is unrealistic to expect the autonomous driving system to recognize all road environments and handle every traffic situation. Models for autonomous driving need to be updated in real time in order for the system to adapt to more situations. This is where online continual learning becomes crucial. Online continual learning is an important method in the field of autonomous driving, as it enables models to update their parameters with streaming input data for adapting to new environments and conditions. Online continual learning in the field of autonomous driving faces several challenges: inefficient data fusion, catastrophic forgetting, insufficient computational resources, violation of road privacy and categories imbalance. To tackle these challenges, we propose an Analytic Online Continual Learning method for 3D Point Cloud Classification (3D-AOCL). This approach utilizes Analytic Learning to update parameters and integrates a feature fusion module along with a category balancer to address the above issues. It is capable of fusing data in feature level, balancing samples across various categories and updating parameters by calculating the analytical solution. We have validated our method on the vehicle side, the infrastructure side, and vehicle-infrastructure cooperative data on the V2X-Seq dataset. The experimental results demonstrate that our model effectively addresses key issues in online continual learning for autonomous driving systems, outperforming other models by approximately 4.00% to 6.00% in AMCA scores while only keeping 0.75% trainable parameters.

1. Introduction

Autonomous driving [1–3] has garnered significant attention in recent years, and its development is progressively transforming the way people travel. In the realm of autonomous driving, point cloud [4–6] data plays a crucial role. Autonomous driving systems integrate point cloud data from multiple sensors, including those on the vehicle and in the infrastructure, to make accurate decisions [7,8]. Compared to traditional image data, point cloud offers more precise and comprehensive environmental information, providing autonomous driving systems with a more accurate basis for decision-making. Point cloud data can represent the position and shape of objects in three-dimensional space and perceive depth information and occlusions in the surrounding environment [9], thereby enhancing the safety and reliability of autonomous driving.

Online continual learning algorithms are of great significance in the field of autonomous driving.[10] Since autonomous driving systems need to handle various unpredictable situations, traditional offline learning algorithms cannot meet the requirements of real-time updates. Online continual learning algorithms, however, can collect data and update models in real-time, allowing autonomous driving systems to learn more information in different scenarios and improve the robustness of the models.

Online continual learning aims to rapidly adapt to new data as it arrives, requiring models to learn from individual or batches of data incrementally and update and refine their knowledge in real-time [11]. Online continual learning in autonomous driving system remains a challenging task due to several difficulties.

- Models are prone to catastrophic forgetting of previously learned data knowledge.
- Utilizing historical training samples may infringe on privacy.

* Corresponding author.
E-mail addresses: zqzeng@scut.edu.cn (Z. Zeng), wiwjwilliam@mail.scut.edu.cn (J. Wang), 202164020379@mail.scut.edu.cn (L. Wu), wi_wklu2452@mail.scut.edu.cn (W. Lu), hpzhuang@scut.edu.cn (H. Zhuang).

https://doi.org/10.1016/j.aej.2024.10.037
Received 22 July 2024; Received in revised form 9 September 2024; Accepted 8 October 2024
Available online 29 October 2024
1110-0168/© 2024 The Authors. Published by Elsevier B.V. on behalf of Faculty of Engineering, Alexandria University. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

- Models need to respond and learn new knowledge quickly under limited computational resources.
- The imbalance of streaming data can cause model parameters to bias towards majority classes.

The main branches of online continual learning [12], including regularization and sample replaying, address some of these issues to varying extents. Catastrophic forgetting [13] refers to the phenomenon where models lose previously learned knowledge when trained on new data. Regularization-based algorithms can alleviate the issue of catastrophic forgetting, but after training on a large amount of new data, catastrophic forgetting may still occur. Sample replaying based algorithms alleviate the forgetting problem by preserving representative samples but risk violating privacy.

Continual learning methods based on analytical learning [14] represent a new branch of continual learning. Analytical learning can overcome the impact of catastrophic forgetting without retaining samples. However, the training approach of analytical learning is not well-equipped to handle the issue of category imbalance. Category imbalance [15] occurs when certain categories have significantly fewer data points compared to others, due to skewed data distributions. This imbalance can lead learners to over-classify the majority class, as it has a higher prior probability in the training data. As a result, the model may be overfitting to the majority classes, leading to severe performance degradation. Additionally, these branches of online continual learning have not shown significant effects [16] when applied to point cloud models, lacking targeted design for point cloud data. Futhermore, point cloud data for autonomous driving can be simultaneously collected from both the infrastructure and vehicle sides. However, existing data fusion schemes, such as Early Fusion, Late Fusion, and Intermediate Fusion, are inadequately designed for point cloud data, as discussed in Section 2.1.

We propose an **A**nalytical **O**nline **C**ontinuous **L**earning method for **3D** point cloud Classification (**3D-AOCL**) that alleviates the aforementioned problem. It overcomes catastrophic forgetting without preserving samples and mitigates the issue of data imbalance, enhancing model performance. Our main contributions are summarized as follows:

- To address the data imbalance problem in continual learning, we propose two strategies: Distance-Based Sampling **(DBS)** or Balanced Analytical Learning **(BAL)**. Both strategies can effectively address the imbalance issue.
- We have designed a feature-level fusion scheme for vehicle-infrastructure cooperative data. This scheme involves extracting features from point cloud data at both the vehicle and infrastructure side, followed by fusion and compression using a feature fusion module.
- We have proposed the 3D point cloud online continual learning classification task based on the V2X-Seq [8] dataset. Our model's capabilities have been confirmed through validation on vehicle side data, infrastructure side data, and vehicle road cooperative data.

The structure the paper is organized as follows. In Section 2, we review the literature relevant to our topic. In Section 3, we describe the implementation details of our method. In Section 4, we demonstrate the effectiveness of our method through validation experiments on the 3D point cloud online continual learning classification task. Finally, in Section 5, we summarize the content of the article.

2. Related works

2.1. Data fusion in vehicle road cooperation system

The key of vehicle road cooperation Systems lies in information sharing, where infrastructure side data aids in vehicle perception and decision-making through data fusion. There are three forms of data fusion at the vehicle side and infrastructure side [17].

Early Fusion achieves fusion by unifying the sensor data from both vehicle and infrastructure sides into a common coordinate system. This approach offers high precision but requires a large amount of data to be transmitted.

Late Fusion shares detection results, meaning that models are deployed at both sides of infrastructure and vehicle, and the computed results are then fused. This method reduces communication overhead but it is limited in terms of accuracy.

Intermediate Fusion shares information at the feature level, where models are used to extract features at both the vehicle and road sides, followed by feature fusion. This approach can reduce communication overhead while maintaining high precision. However, no prior work has explored Intermediate Fusion on point cloud data.

2.2. Point cloud dataset and classification

Point cloud data are essential for perceiving surroundings and making accurate decisions in autonomous driving systems. The ModelNet40 [18] dataset, a widely used benchmark for point cloud classification, contains 12,311 CAD-generated objects across 40 categories (e.g., airplane, car, lamp). However, ModelNet40 includes various objects that go beyond real-world road environments and are not specifically designed for autonomous driving applications. Early point cloud datasets for autonomous driving, such as Kitti [19] and CamVid [20], are collected from real-world environments but contain a limited variety of objects. Later, datasets like IDDA [21] and GTA-V [22] use simulation tools like OpenCDA [23] and SUMO [24] to generate synthetic samples, increasing the variety of objects. Other datasets, such as Zensact [25], Waymo Motion [26], and ONCE [27], expand datasets by collecting large-scale real-world point cloud data for autonomous driving. However, these datasets only include single-view data captured by sensors on the vehicle side. In practice, autonomous driving systems often rely on multi-view data from both vehicle and infrastructure sides. The V2X-Sim [28] dataset provides small-scale, simulated point cloud data collected simultaneously from both vehicle and infrastructure perspectives. Building on this, the V2X-Seq [8] dataset collects large-scale, real-world sequential point cloud data, making it the first large-scale sequential point cloud dataset captured entirely from real-world environments. V2X-Seq includes data from paired sensors on both vehicle and infrastructure sides, offering an ideal benchmark for point cloud classification in autonomous driving systems.

Point cloud classification presents unique challenges compared to image classification. Point clouds are represented by coordinates (x, y, z) with potential additional attributes like color or intensity. Unlike images, Point clouds are often sparse and incomplete, making classification more challenging. Additionally, point clouds are inherently unordered, requiring the consideration of local neighborhoods for feature extraction rather than isolated point analysis [29,30]. Specialized techniques are necessary to capture the local structural information in point cloud data. The PointNet model [31] addresses these challenges by employing max pooling as an aggregation function to symmetrically integrate information from each point, thereby accommodating the unordered nature of point clouds. Expanding on PointNet, PointNet++ [32] introduces a hierarchical feature learning structure that extracts both global and local information from point clouds in a hierarchical manner. This approach further enhances feature extraction capabilities through sampling and aggregation, maximizing the retention of the 3D structure within point clouds.

2.3. Online continual learning

Continual learning [12] refers to the ability of intelligent systems to adapt to external changes, continuously acquiring, updating, accumulating, and utilizing knowledge. Online learning [11,33] requires models to respond rapidly to new incoming data and to learn new knowledge. As the model continuously acquires new information, it can adapt to new environments, making it extensively applicable in the field of autonomous driving. Online continual learning, however, is subject to more stringent requirements and faces several major challenges, such as limited storage, low efficiency of real-time model updates, and catastrophic forgetting. To address these issues, researchers have proposed many solutions.

Regularization-Based Continual Learning. These methods for continual learning seek to integrate regularization terms that balance the learning of new parameters with the preservation of old knowledge, thereby mitigating catastrophic forgetting. Notable approaches in this domain include Learning Without Forgetting (LWF) [34], which freezes old model parameters and utilizes soft label outputs as regularization in the loss function to maintain the parameter distribution of previously learned tasks. Elastic Weight Consolidation (EWC) [35] adds regularization terms to the loss function to track parameter importance and minimize updates to critical parameters based on assigned weights. Relying solely on regularization terms in the loss function can only preserve old knowledge for a limited period, and catastrophic forgetting can still occur during continuous training. However, real-world autonomous driving systems require models to continually learn from incoming data, and they continue to suffer from catastrophic forgetting even when using regularization-based continual learning methods.

Replay-Based Continual Learning. The replay-based continual learning method allows the model to store a small number of training samples and use old samples to balance the performance of old and new tasks when training new tasks. Incremental Classifier and Representation Learning(iCaRL) [36] maintains a memory bank that stores representative samples for each class. Mnemonics [37] continuously optimizes old examples to adapt to new data distributions by dividing old examples into multiple subsets and alternately using them as validation sets. Although replay-based methods can preserve old knowledge indefinitely, autonomous driving systems need to store old samples and retrain models on them, which introduces additional memory and computational resource requirements. Furthermore, permanently storing old data raises privacy concerns, as it could be vulnerable to theft by malicious attackers.

Historical Sample-Generated Continual Learning. In the realm of image continual learning, numerous attempts have been made to employ Generative Adversarial Networks (GANs) to generate images and counteract catastrophic forgetting [38,39]. However, the inherent noise introduced by generative models, combined with the lack of mature solutions in the point cloud generation domain, renders them less applicable for the tasks under consideration. Point cloud data for real-world autonomous driving systems are more complex and varied, making their generation more challenging. Low-quality generated samples will significantly degrade model performance.

Analytic Learning-Based Continual Learning. Analytic Class-Incremental Learning (ACIL) [14] represents a novel direction in continual learning, derived from pseudo-inverse learning [40]. The core principle involves maintaining a fixed feature extractor while transforming the model's classifier into dimensionality-increasing hidden layer and a fully connected classification layer, as shown in Fig. 2. The dimensionality-increasing hidden layer transforms the input features from a low-dimensional space to a higher-dimensional one. The fully connected classification layer then maps these high-dimensional features to their corresponding class labels. By storing information via the autocorrelation matrix, ACIL retains the ability to recall old tasks and acquires analytical solutions through pseudo-inverse matrices using the least squares method. This approach offers advantages over traditional backpropagation methods, such as reduced memory consumption and simplified computational processes. In real-world autonomous driving environments, which are resource-limited, analytic learning-based methods are more suitable.

3. Method

3.1. Overview

The pipeline of our method 3D-AOCL is shown in Fig. 1 and includes the following modules:

Point Cloud Preprocessing Module normalizes the point cloud data to the uniform point count through upsampling or downsampling.

Feature Extraction and Fusion Module involves converting the raw point cloud data into feature representations and fusing features if multiple point clouds from different sensors are available.

Classification and Online Continual Learning Module employs a multi-layer perceptron (MLP) as the classifier to categorize the fused features and utilizes analytical learning method to continuously update the classifier with streaming training data.

Category Balancing Module addresses the category imbalance problem by Distance-Based Sampling (DBS) or Balanced Analytical Learning (BAL), ensuring that the model does not overly focus on the majority classes during training.

Details of the above modules will be presented in the following sections.

3.2. Point cloud preprocessing module

Point cloud data from different sensors may contain varying numbers of points and need to be normalized to a uniform point count for subsequent feature extraction and fusion. Consider our input point cloud data as $P = \{p_1, p_2, \dots, p_n\}$, where n represents the number of points in the point cloud P, t is the desired uniform point count. If $n \geq t$, we apply the Farthest Point Sampling (FPS) [41,42] algorithm to reduce the point cloud to exactly S points. FPS algorithm prefers to select data points at the far end and ignore data points at the near end, with the aim of preserving the overall geometric structure of the point cloud. We calculate the distance between each data point and the center of the point cloud, then sort the data points in descending order based on these distances. The top t data points are selected to compose the point cloud.

If $n < t$, we firstly employ the interpolation based upsampling algorithm to increase the point count to exceed S and then utilize FPS to sample S data points. The interpolation is achieved using a polynomial fitting technique. For a point p, we denote $N_p(p, k)$ as the set of k nearest neighbors of p, including p itself. We use the polynomial function to fit the distribution of data in $N_p(p, k)$. The polynomial function can be expressed as:

$$poly(x_i, y_i) = \beta_0 + \beta_1 x_i + \beta_2 y_i + \beta_3 x_i^2 + \beta_4 y_i^2 + \cdots, \tag{1}$$

where β represents the coefficients of the polynomial, x_i and y_i are the x-coordinate and y-coordinate of ith point in $N_p(p, k)$. The parameter β is obtained by optimizing the objective function as follows:

$$S(\beta) = \sum_{(x_i, y_i) \in N_i} \left(z_i - poly(x_i, y_i)\right)^2, \tag{2}$$

where z_i is the z-coordinate of the ith point in N_p. Once $poly(x_i, y_i)$ is determined, we sample new coordinate points from the polynomial surface to augment the original point cloud. This process is repeated until the point number of point cloud exceeds S. The details of point cloud preprocessing is shown in Algorithm 1.

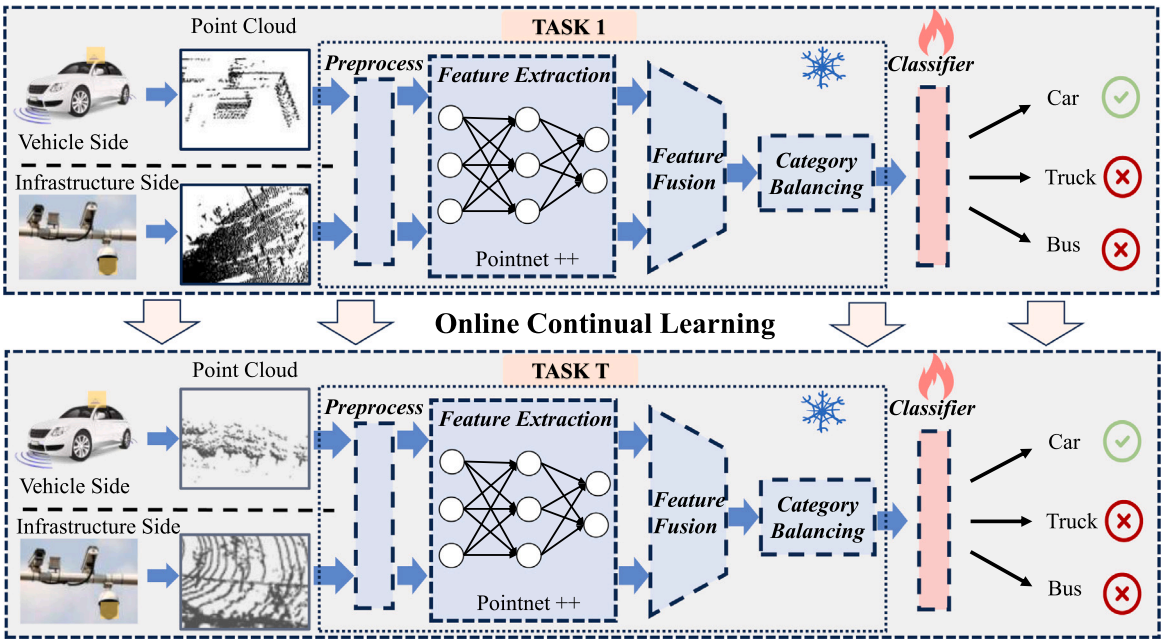

Fig. 1. The structure of 3D-AOCL contains the following part: (i). The point cloud preprocessing module normalizes the point cloud data to uniform point count. (ii). The feature extraction module based on Pointnet++ extracts feature vector from point cloud data. (iii). The feature fusion module fuses the feature vectors from the vehicle side and the infrastructure side. (iv). The category balancing module utilizes distance-based sampling (DBS) or balanced analytical learning (BAL) to solve the imbalance problem. (v). The classifier module categories the point cloud data according to the feature vectors. All modules are fixed, except for the classifier module. We continuously update the parameters of the classifier module through analysis learning.

Algorithm 1 Point Cloud Preprocessing

Input: P is the point cloud; p_i is the i-th point in P; t is the desired uniform point count; n is the number of point in P; for polynomial fitting, we consider the k nearest neighbors. N_i is the k nearest neighbors set of p_i.

POLYNOMIALFIT(N_i):
  $poly(x_i, y_i) = \beta_0 + \beta_1 x_i + \beta_2 y_i + \beta_3 x_i^2 + \beta_4 y_i^2 + \ldots$
  $S(\beta) = \sum_{(x,y)\in N_i} \left(z_i - poly(x_i, y_i)\right)^2$
  $\beta \leftarrow$ MINIMIZE($S(\beta)$)
  return $poly(x_i, y_i)$
if $n < t$ **then**
  while $n < t$ **do**
    $P_{new} = []$
    for p_i in P **do**
      $poly(x_i, y_i) =$ POLYNOMIALFIT(N_i).
      Sample (x_{new}, y_{new}, z_{new}) from $poly(x_i, y_i)$.
      Add (x_{new}, y_{new}, z_{new}) into P_{new}.
    end for
    $P = P + P_{new}$.
  end while
end if
Calculate the center c of P.
Sort P according to the distance to c.
P_N = P[0:t].

Output: The normalized point cloud P_N with uniform point count.

3.3. Feature extraction and fusion module

We utilized the pretrained PointNet++ model excluding the last linear layer as the feature extractor and all its parameters are frozen. The parameters are represented by W_{backbone} and the feature extraction process can be represented as:

$$F_P = f(P_N, W_{\text{backbone}}), \tag{3}$$

where F_P is the extracted feature of the normalized point cloud P_N. Suppose F_{veh} represents the extracted features from point cloud data collected by sensors on the vehicle side, and F_{inf} represents the extracted features from the infrastructure side. We first make alignment to both features and then perform fusion by addition, which can be expressed as:

$$F_{fusion} = F_{veh} W_{veh} + F_{inf} W_{inf}, \tag{4}$$

where W_{veh} and W_{inf} are the alignment transformation matrices for the vehicle side and the infrastructure side, respectively.

3.4. Classification and online continual learning module

As depicted in Fig. 2, we construct a multiple layer perception to category the fusion features, which can be expressed as:

$$\mathbf{A} = \text{ReLU}(F_{fusion} \cdot W_{in}). \tag{5}$$

$$\mathbf{C} = \mathbf{A} \cdot W_{out}, \tag{6}$$

where W_{in} is the parameter of the hidden layer, W_{out} is the parameter of the classification layer, $\mathbf{A} \in \mathbb{R}^h$ is the activation vector and $\mathbf{C} \in \mathbb{R}^c$ is an one-hot vector representing the classification result (see Fig. 2).

During online continual learning, training data arrives in a streaming fashion, requiring the model to be updated in real time. Especially, we only update the classifier during online continual learning and keep other module fixed. Suppose the stream train data can be divide into m part and each part can be defined as one task for classifier training. For each task T_t, the training data can be represented by $D_t = \{(P_t^1, Y_t^1), (P_t^2, Y_t^2), \ldots, (P_t^m, Y_t^m)\}$, where P_t^i are the ith point cloud data in D_t and Y_t^i are the corresponding class label set.

Our training method for online continual learning adopts an analytical learning approach [14]. The analytical learning randomly initialize

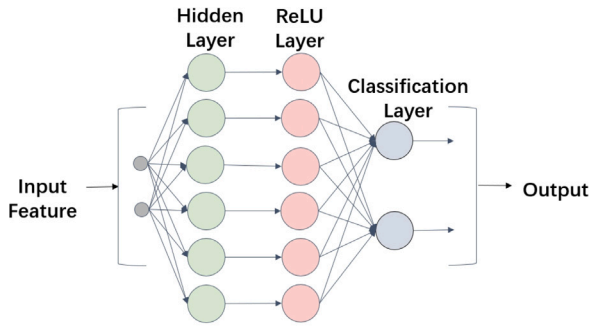

Fig. 2. The structure of the classifier consists of three layers: the first layer is a hidden layer, the second layer is a rectified linear unit (ReLU) layer, and the third layer is a classification layer. ReLU function outputs 0 for negative inputs and return the same value for positive inputs. Only the third layer updates its parameters during training.

W_{in} and obtains W_{out} by calculating the analytical solution of linear regression. Supposing the weight of the classification layer after training on D_t is $W^t_{out} \in \mathbb{R}^{h\times c}$, which can be obtained by solving:

$$\hat{\boldsymbol{W}}^t_{out} = \underset{\boldsymbol{W}^t_{out}}{\operatorname{argmin}} \left(\|\boldsymbol{Y}_{\mathbf{1}:t} - \mathbf{A}_{\mathbf{1}:t}\boldsymbol{W}^t_{out}\|^2_{\mathrm{F}} + \gamma\|\boldsymbol{W}^t_{out}\|^2_{\mathrm{F}} \right), \tag{7}$$

where γ is the regularization parameter, c is the number of categories, $\mathbf{A}_{\mathbf{1}:t}$ represents the activation vectors of point cloud from historical task data, including task t itself, and $\boldsymbol{Y}_{\mathbf{1}:t}$ denotes corresponding class labels. According to the Least Squares Algorithm, the optimal solution $\boldsymbol{W}^t_{out}$ is:

$$\hat{\boldsymbol{W}}^t_{out} = \left(\sum_{i=1}^{t} \mathbf{A}_i^\top \mathbf{A}_i + \gamma \boldsymbol{I} \right)^{-1} \left(\sum_{i=1}^{t} \mathbf{A}_i^\top \boldsymbol{Y}_i \right). \tag{8}$$

We denote the autocorrelation matrix $\boldsymbol{R}_t$ as $\left(\sum_{i=1}^{t} \mathbf{A}_i^\top \mathbf{A}_i + \gamma \boldsymbol{I}\right)^{-1}$ and then Eq. (8) can be:

$$\hat{\boldsymbol{W}}^t_{out} = \boldsymbol{R}_t \cdot \sum_{i=1}^{t} \mathbf{A}_i^\top \boldsymbol{Y}_i. \tag{9}$$

The weight matrix $\hat{\boldsymbol{W}}^t_{out}$ and the autocorrelation matrix $\boldsymbol{R}_t$ can be updated iteratively as following:

$$\boldsymbol{R}_t = \boldsymbol{R}_{t-1} - \boldsymbol{R}_{t-1}\mathbf{A}_t^\top(\boldsymbol{I} + \mathbf{A}_t\boldsymbol{R}_{t-1}\mathbf{A}_t^\top)^{-1}\mathbf{A}_t\boldsymbol{R}_{t-1}, \tag{10}$$

$$\hat{\boldsymbol{W}}^t_{out} = (\boldsymbol{I} - \boldsymbol{R}_t\mathbf{A}_t^\top\mathbf{A}_t)\hat{\boldsymbol{W}}^{t-1}_{out} + \boldsymbol{R}_t\mathbf{A}_t^\top\boldsymbol{Y}_t. \tag{11}$$

We initialize $\hat{\boldsymbol{W}}^0_{out}$ and $\boldsymbol{R}_0$ as:

$$\hat{\boldsymbol{W}}^0_{out} = \mathbf{0}_{h\times c}, \tag{12}$$

$$\boldsymbol{R}_0 = \frac{1}{\gamma}\boldsymbol{I}. \tag{13}$$

In practice, Eq. (11) performs multiple matrix multiplications, which can increase memory occupation, so it can also be computed as to save memory:

$$\hat{\boldsymbol{W}}^t_{out} = \boldsymbol{R}_t\mathbf{A}_t^\top(\boldsymbol{Y}_t - \mathbf{A}_t\boldsymbol{W}^{t-1}_{out}). \tag{14}$$

By iteratively updating model parameters according to Eqs. (10) and (19), our approach eliminates the need for backpropagating loss and computing gradients for each parameter, which is both time- and memory-intensive. Moreover, as noted by [43], gradients can potentially leak private information from training samples. The analytical approach bypasses gradient descent, significantly reducing the risk of privacy leakage.

3.5. Category balancing module

In this section, we will introduce how to address the problem of class imbalance. Current methodologies to address class imbalance in continual learning primarily rely on sampling-based approaches, such as upsampling minority classes or downsampling majority classes [36, 44]. However, point cloud data is fundamentally different from other data formats, making it challenging to apply these existing sampling-based methods directly. New sampling techniques specifically designed to handle class imbalance in point cloud data are yet to be explored. Additionally, analytic continual learning presents a different paradigm from traditional continual learning methods, suggesting the potential for novel approaches beyond sampling to address class imbalance in this context.

We have designed two strategies to tackle this issue, Distance-Based Sampling (**DBS**) and Balanced Analytical Learning (**BAL**). DBS involves performing resampling of the minority class on the fused features. BAL achieves balanced analytical learning by improving the training formula of analytical learning.

Distance-base sampling (DBS) is derived from the iCaRL algorithm [36]. Samples closer to the mean data are more likely to be representative. We duplicate these representative samples in the minority class to address the class imbalance problem.

The original representation of point cloud data is redundant. Therefore, we calculate distances based on the extracted features of the point cloud and perform sampling at the feature level to reduce the computational workload. We compute the feature means μ_c within the feature set F_c for category c as follows:

$$\mu_c = \frac{1}{m}\sum_{F\in F_c} F, \tag{15}$$

where m is the count of features in F_c. The distance between each feature sample F and μ_c is calculate as:

$$d(F, \mu_c) = \sqrt{\sum_{j=1}^{v}(F_j - \mu_{cj})^2}, \tag{16}$$

where $F, \boldsymbol{\mu_c} \in \mathbb{R}^v$. We sort the calculated distances in ascending order. From the distance list we select a certain proportion α of F_c as the candidate set F_a. Then we add F_a into F_c repeatedly until the feature amount of category catch up with the majority class. Finally, we obtain the resampling dataset F^{re}_c for category c.

Balanced Analytical Learning (BAL) solves the class imbalance problem by assign dynamic weights for different category when update the classifier parameter using Eqs. (10), (11).

Due to category imbalance, the majority class will be assigned higher attention during training, causing the model to tend to classify features as the majority class. Based on this understanding, we consider adding dynamic balancing weights to the objective function (7) as:

$$\hat{\boldsymbol{W}}^t_{out} = \underset{\boldsymbol{W}^t_{out}}{\operatorname{argmin}} \left(\boldsymbol{\sigma}_{\mathbf{1}:t}\| \left(\boldsymbol{Y}_{\mathbf{1}:t} - \mathbf{A}_{\mathbf{1}:t}\boldsymbol{W}^t_{out}\right)\|^2_{\mathrm{F}} + \gamma\|\boldsymbol{W}^t_{out}\|^2_{\mathrm{F}} \right), \tag{17}$$

where $\boldsymbol{\sigma}_{\mathbf{1}:t} = [\boldsymbol{\sigma}_1, \boldsymbol{\sigma}_2, \ldots, \boldsymbol{\sigma}_t]$ is all weight vectors for historical tasks, $\boldsymbol{\sigma}_t = [\boldsymbol{\sigma}_{t,1}, \boldsymbol{\sigma}_{t,2}, \ldots, \boldsymbol{\sigma}_{t,c}]$ is the weight vector for task t, $\boldsymbol{\sigma}_{t,i}$ is the weight for class c in task t. We calculate $\boldsymbol{\sigma}_{t,i}$ as:

$$\boldsymbol{\sigma}_{t,i} = \frac{max(count(D^1_t), \ldots, count(D^c_t))}{count(D^i_t)}, \tag{18}$$

where $count(D^i_t)$ is the data amount of class i in the training data of task t. The expression for parameter $\hat{\boldsymbol{W}}^t_{out}$ updating is:

$$\hat{\boldsymbol{W}}^t_{out} = (\boldsymbol{I} - \boldsymbol{R}_t\boldsymbol{\sigma}_t \cdot \mathbf{A}_t^\top\mathbf{A}_t)\hat{\boldsymbol{W}}^{t-1}_{out} + \boldsymbol{R}_t\boldsymbol{\sigma}_t \cdot \mathbf{A}_t^\top\boldsymbol{Y}_t, \tag{19}$$

The updating expression for $\boldsymbol{R}_t$ is:

$$\boldsymbol{R}_t = \boldsymbol{R}_{t-1} - \boldsymbol{R}_{t-1}\boldsymbol{\sigma}_t \cdot \mathbf{A}_t^\top(\boldsymbol{I} + \mathbf{A}_t\boldsymbol{R}_{t-1}\boldsymbol{\sigma}_t \cdot \mathbf{A}_t^\top)^{-1}\mathbf{A}_t\boldsymbol{R}_{t-1}. \tag{20}$$

And Eq. (19) can be simplified as:

$$\hat{\boldsymbol{W}}^t_{out} = \boldsymbol{R}_t\boldsymbol{\sigma}_t \cdot \mathbf{A}_t^\top(\boldsymbol{Y}_t - \mathbf{A}_t\hat{\boldsymbol{W}}^{t-1}_{out}). \tag{21}$$

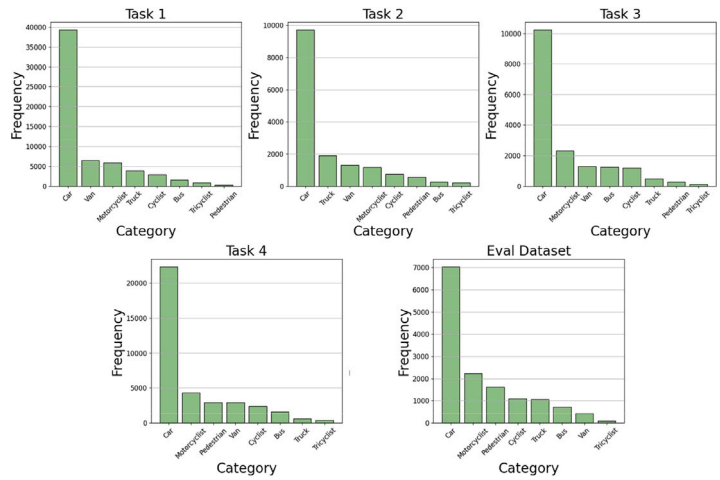

Fig. 3. This is a statistical chart of the categories with data quantities, which clearly shows that the dataset is extremely imbalanced.

4. Experiments

In this section, we will introduce how we validated the effectiveness of the 3D-AOCL on the 3D point cloud online continual learning classification task based on V2X-Seq dataset, including the content of: experiments setup, main results, compared methods, hyperparameter analysis and case study.

4.1. Experiments setup

Datasets. ModelNet40 [18] dataset is used for point cloud classification and contains 40 object categories. Most point clouds in ModelNet40 have approximately 100 points, which is relatively sparse and makes it challenging to extract effective information. To address this, We augment the point cloud data in ModelNet40 by first randomly sampling the point cloud and then using the upsampling method described in 3.2 to obtain 512 points. The processed ModelNet40 dataset is used to pre-train our feature extraction backbone.

The V2X-Seq [8] dataset is a large-scale, real-world dataset for testing autonomous driving systems. It contains 3D point cloud data collected simultaneously from paired sensors on the vehicle side and the infrastructure side over a period of time. Each point cloud dataset is annotated with 3D detection targets, encompassing 8 different types. Following by the data partitioning strategy of CLAD-C [45], we sort the point cloud data by time and extract every 15 frames as one training task, constructing a total of 4 training tasks. The remaining data is utilized as the test set. For each frames data, we extract the point clouds of the annotated 3D bounding boxes along with their type annotations, which are used for the 3D point cloud data classification task. We exclude any boxes with a point count less than 64, as they contain insufficient 3D information. We present the category distribution of our training tasks and evaluation dataset in Fig. 3. It can be clearly observed that the V2X-Seq dataset suffers from a significant class imbalance issue.

Backbone and Pretraining Process. We pretrain the SSG version of PointNet++ [32] on the ModelNet40 dataset for 40 epochs and use the model, excluding the last classifier layer, as our feature extraction module. After pretraining the feature extraction module, we add a feature fusion module along with a new classifier layer. We then pretrain the feature fusion module and the classifier layer on D_1 (training data of task 1) for 40 epochs. Subsequently, we retain only the feature fusion module and discard the classifier layer. We then add and initialize a new classifier as described in Section 3.4 for subsequent online continual learning.

Evaluation Metric. We employ the Average Mean Class Accuracy (AMCA) as our evaluation metric, which is used by CLAD-C [45] to measure a model's classification accuracy in online learning. The AMCA metric is computed as:

$$\mathrm{AMCA} = \frac{1}{|T| \cdot C} \sum_{t \in T} \sum_{c=1}^{C} Acc_t^c, \tag{22}$$

where Acc_t^c represents the accuracy of class c after training on task T_t, $|T|$ represents the number of tasks and C represents the number of categories. We evaluate the scenarios where only vehicle-side data is available (AMCA_V), only infrastructure-side data is available (AMCA_I), and where both are available (AMCA_B). Additionally, we will track the average accuracy after each training task (Acc_t) to observe changes in accuracy. We also present the number of training parameters (TP), which reflects the resource consumption and algorithm efficiency.

Hyperparameters. The desired point count of each point cloud t is 512. The hidden state size h is set to 1500. γ for the regularization term in Eq. (17) is 100. The sampling proportion α for DBS is set to 0.8.

4.2. Comparison methods

Without CL trains the model using conventional training methods without any continual learning strategies.

FT fixes the feature extraction and fusion modules, and only fine-tunes the classifier.

LWF [34] **and iCaRL** [36] represent regularization-based CL and sample replaying-based CL, respectively.

ACIL [14] employs analytical continual learning without addressing the category imbalance problem.

DBS/BAL means we utilize different strategies to addressing the category imbalance problem.

4.3. Major result

The results obtained using vehicle-side only data or infrastructure-side only data are presented in Table 1. The results obtained using vehicle-infrastructure cooperation data are presented in Table 2. The performance trends of various baselines after sequential training across four tasks are illustrated in Fig. 4.

As depicted in Tables 1 and 2, when a model is trained without any online continual learning methods, referred to as Without CL and FT methods, it struggles to effectively retain memory, leading to catastrophic forgetting and performance degradation. As depicted in Fig. 5, Without CL method resulted in a decrease in AMCA after sequential training on multiple tasks, regardless of whether vehicle-side

Table 1
The performance of different baselines on vehicle-side only and infrastructure-side only data. The best results are marked in bold, and the second-best results are underlined.

Method	TP	Vehicle-side only					Infrastructure-side only				
		$AMCA_V$	Acc_1	Acc_2	Acc_3	Acc_4	$AMCA_I$	Acc_1	Acc_2	Acc_3	Acc_4
Without CL [32]	1.6M	45.05	45.24	38.84	48.14	47.98	50.12	46.02	36.12	64.15	54.19
FT	**1.8K**	47.50	38.72	47.58	50.82	52.88	48.43	38.89	52.16	53.26	49.41
LWF [34]	1.6M	50.50	39.02	46.97	57.12	58.89	54.90	44.83	51.56	67.22	55.99
iCaRL [36]	1.6M	47.60	28.87	51.94	54.96	54.63	44.05	20.54	49.86	51.03	54.77
ACIL [14]	12.0K	48.93	43.87	45.23	51.84	54.78	46.35	37.25	48.65	50.64	48.86
Ours(DBS)	12.0K	54.38	48.83	51.78	56.02	60.89	**58.77**	**47.23**	**58.56**	64.02	65.27
Ours(BAL)	12.0K	**56.48**	**50.88**	**52.73**	**60.05**	**62.26**	58.64	45.52	56.45	**64.53**	**68.06**

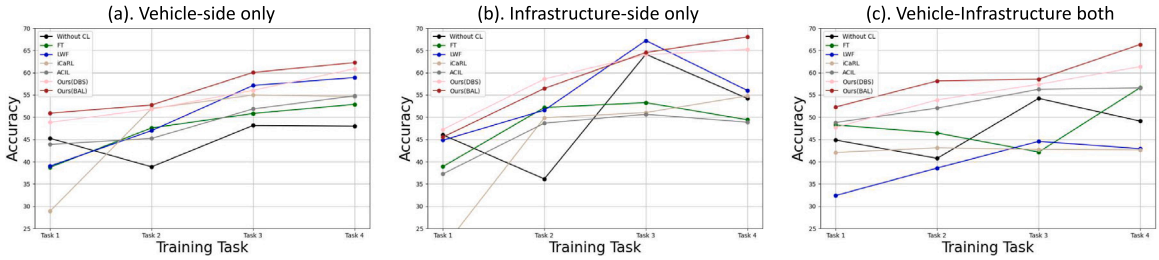

Fig. 4. Performance variations of different baselines after sequential training on four tasks: (a) when only vehicle-side data is available, (b) when only infrastructure-side data is available, and (c) when both vehicle and infrastructure data are available.

Table 2
The performance of different baselines on vehicle-infrastructure cooperation data. The best results are marked in bold, and the second-best results are underlined.

Method	TP	Vehicle-infrastructure cooperation				
		$AMCA_B$	Acc_1	Acc_2	Acc_3	Acc_4
Without CL [32]	1.6M	47.23	44.86	40.74	54.21	49.11
FT	**1.8K**	48.37	48.26	46.44	42.16	56.62
LWF [34]	1.6M	39.61	32.39	38.57	44.56	42.92
iCaRL [36]	1.6M	42.65	42.07	43.12	42.76	42.65
ACIL [14]	12.0K	53.41	48.80	52.03	56.26	56.55
Ours(DBS)	12.0K	55.07	47.71	53.87	57.38	61.33
Ours(BAL)	12.0K	**58.82**	**52.28**	**58.14**	**58.52**	**66.34**

data only, infrastructure-side data only, or both were used. This decline is likely due to the model forgetting previously acquired knowledge. FT method also led to a decrease when using infrastructure-side data only and vehicle-infrastructure data combined, except when using vehicle-side data only, possibly due to the limited number of training tasks. In contrast, models trained with online continual learning methods showed improved performance during sequential training of new tasks.

As shown in Table 1, our method, 3D-AOCL, demonstrates superior performance with both the DBS and BAL strategies, achieving AMCA scores of 54.38% and 56.48% in vehicle-side experiments, and 58.77% and 58.64% in infrastructure-side experiments, respectively. For vehicle-infrastructure cooperation experiments, 3D-AOCL achieves AMCA scores of 55.49% and 58.82%, as detailed in Table 2. In the vehicle-side-only data setting, 3D-AOCL (BAL) surpasses the best baseline, LWF, by 5.98%. In the infrastructure-side-only data setting, 3D-AOCL (DBS) outperforms the best baseline, LWF, by 3.87%. When both vehicle and infrastructure data are available, 3D-AOCL (BAL) exceeds the best baseline, ACIL, by 5.41%. In summary, our method achieves approximately 4.00%–6.00% higher AMCA compared to other models, as illustrated in Fig. 5. As depicted in Figs. 4 and 5, our method outperforms CL-based approaches, such as LWF and iCaRL, due to its superior ability to address the catastrophic forgetting problem. Additionally, as shown in Fig. 4, the results for ACIL are notably poorer than ours, especially when only infrastructure-side data is available. This is likely due to ACIL's inability to effectively handle the extreme category imbalance in the training data.

In terms of efficiency, our method requires significantly fewer trainable parameters than other methods, except for FT. As shown in Table 1,our method requires only 12.0K trainable parameters, which is just 0.75% of the full model's 1.6M parameters. It introduces a comparable number of trainable parameters to the FT method, resulting in shorter training times, as shown in the line chart in Fig. 5. Furthermore, because our analytical learning method does not require gradient propagation, it avoids storing gradients, leading to lower memory consumption. Overall, our method can be updated quickly even with limited computational resources, which aligns with the practical constraints of autonomous driving.

4.4. Hyperparameter analysis

4.4.1. The sampling proportion α in DBS

To investigate the impact of sampling proportion in DBS Algorithm, we evaluation our methods under different sampling proportion α. The relationship between the sampling proportion (α) and the accuracy rate is presented in Table 3.

According to Table 3, we found that when the confidence level is 1, i.e., all features are resampled in a certain proportion, the test results are not as good as when the confidence level is 0.8. This may be due to some noisy samples being duplicated multiple times. When the confidence level α is small, i.e., when a large number of values near the mean of the resampled features are used, it can be seen that the classification performance is even weaker than when the training samples are imbalanced. The reason may be there is a slight deviation between the distribution of the training sample features and the distribution of the test samples. A large number of samples near the feature mean lead to overfitting of the model on the training data.

4.4.2. The category weight σ in BAL

The parameter σ in Eq. (23) is set based on empirical judgment. To validate the reasonableness of this parameter setting, we set the category weight to be the ratio of the maximum class sample count to the class sample count raised to the power of β, and then conduct experiments.

$$\sigma_{t,i} = \left(\frac{max(count(D_t^1), \ldots, count(D_t^c))}{count(D_t^i)} \right)^{\beta}, \tag{23}$$

As shown in Table 4, $\beta = 1$ indicates the BAL algorithm, and $\beta = 0$ indicates ACIL. After training for each task, the performance of BAL

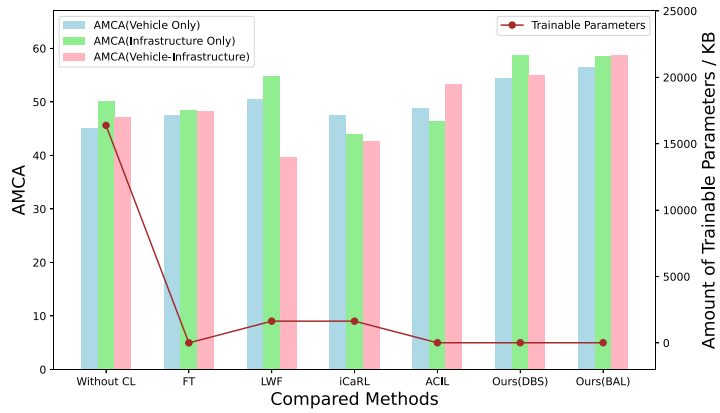

Fig. 5. The Average Mean Class Accuracy (AMCA) results for different baselines, using vehicle data only, infrastructure data only, or both, are displayed in a side-by-side bar chart. The line chart illustrates the number of trainable parameters for each baseline, measured in KB.

Table 3
The relationship between sampling proportion and accuracy at each phase of the task when vehicle-Infrastructure cooperation data is available. The best results are marked in bold.

α	Acc_1	Acc_2	Acc_3	Acc_4	$AMCA_B$
1	43.74	**54.45**	57.27	60.33	53.94
0.9	47.64	51.13	56.71	60.24	53.93
0.8	**47.71**	53.87	**57.38**	**61.33**	**55.07**
0.7	44.18	51.48	52.94	58.54	51.79
0.6	43.80	48.55	51.35	56.39	50.02
0.5	43.73	47.03	53.04	55.41	49.80
0.4	36.15	41.48	47.55	52.21	44.35
0.3	36.44	41.80	49.98	54.41	45.66
0.2	36.21	38.57	46.89	52.19	43.46
0.1	32.26	37.68	43.68	49.65	40.81

Table 4
The relationship between category weight and accuracy at each phase of the task when vehicle-Infrastructure cooperation data is available. The best results are marked in bold.

β	Acc_1	Acc_2	Acc_3	Acc_4	$AMCA_B$
−1	33.65	39.07	37.01	43.72	39.41
0	48.80	52.03	52.34	56.26	53.41
0.5	49.59	54.08	54.71	57.53	55.02
1	**52.28**	**58.14**	**58.52**	**66.34**	**58.82**
1.5	51.99	57.63	57.99	59.99	58.38
2	51.70	52.70	55.59	33.26	45.96
3	23.75	13.69	13.59	12.50	15.21

consistently reaches optimal levels, demonstrating the reasonableness of the w_t element setting in the BAL algorithm.

From the data in the table, we can also infer that when β is less than 1, the model tends to overly focus on the majority classes after training. Conversely, when β is greater than 1, the model overcompensates by excessively focusing on the minority classes. Both situations lead to a decline in performance.

4.5. Case study

To further verify the effectiveness of our methods, we present detailed cases: cases of 3D-AOCL in Fig. 8, cases of non-continual learning based methods in Fig. 6, cases of traditional continual learning based methods in Fig. 7.

As depicted in Fig. 6, non-continual learning methods, represented by the Without CL approach, still suffer from the catastrophic forgetting problem. When provided with infrastructure-side data, the model initially fails to classify correctly and only learns the correct classification after several training tasks. However, with additional training tasks, the model disregards previously learned knowledge and reverts to incorrect

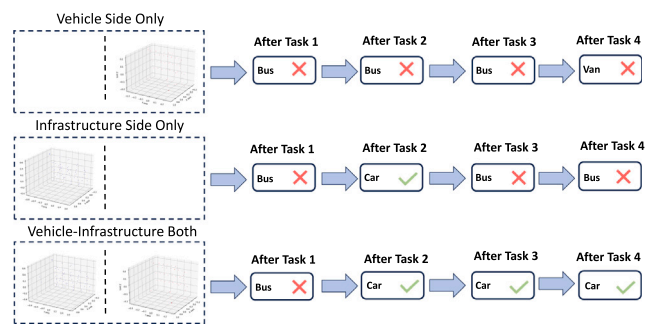

Fig. 6. Several cases of non-continual learning based methods (represented by **Without CL**) for point cloud data classification in autonomous driving systems, using vehicle-side data, infrastructure-side data, or a combination of both, during the continual training of four distinct tasks.

classifications. For traditional continual learning methods, represented by the LWF method, some cases are shown in Fig. 7. Although the model can initially provide correct classifications with infrastructure-side data and learn accurate classifications after a few training tasks when given vehicle-side or vehicle-infrastructure data, these methods can only partially alleviate the catastrophic forgetting problem. For instance, the model retains correct classifications during the training of Task 2 when only infrastructure data is provided. However, all methods ultimately classify the object as a car, primarily due to the class imbalance problem. The car class dominates the training data across all tasks, as shown in Fig. 3.

As depicted in Fig. 8, our method performs well when both vehicle and infrastructure data are available. Even when only vehicle or infrastructure data is provided, our method can still achieve correct classification after continual training. Notably, our method effectively addresses the catastrophic forgetting problem, as the model retains accurate classifications all the time even when trained on additional new tasks.

5. Conclusion and future work

In this paper, we introduce an analytical learning based method for 3D point cloud online continual learning classification 3D-AOCL, which can be applied in autonomous driving systems. By using analytical learning, we have addressed the issue of catastrophic forgetting in online continuous learning, while enhancing the efficiency of model training and reducing the resources required for training. To tackle the problem of categories imbalance, we introduced a category balancing module that mitigates the impact of imbalance. To tackle the challenge of data fusion, we introduced a feature level fusion module which can

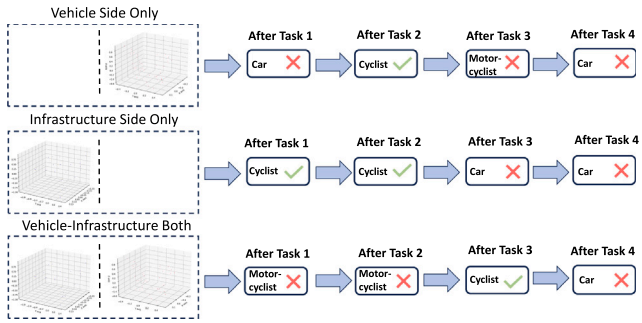

Fig. 7. Several cases of traditional continual learning based methods (represented by **LWF**) for point cloud data classification in autonomous driving systems, using vehicle-side data, infrastructure-side data, or a combination of both, during the continual training of four distinct tasks.

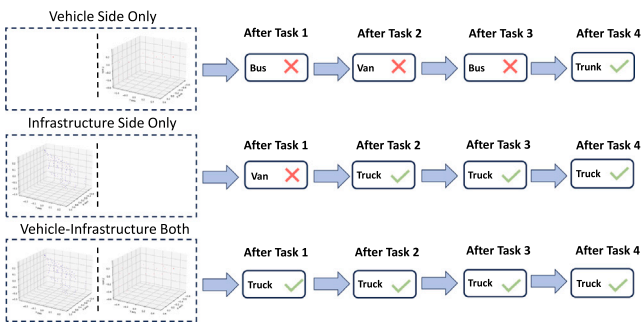

Fig. 8. Several cases of the **3D-AOCL** method for point cloud data classification in autonomous driving systems, using vehicle-side data, infrastructure-side data, or a combination of both, during the continual training of four distinct tasks.

fuse data in the feature level. We conducted experiments on the vehicle side, the infrastructure side, and vehicle-infrastructure cooperative data from the V2X-Seq dataset. Experiment results demonstrate the competitiveness of 3D-AOCL in addressing online continuous learning challenges in autonomous driving systems by surpass other baselines by approximately 4% to 11% in AMCA scores while only keeping 0.07% trainable parameters.

Obviously, 3D-AOCL can be easily extended to fields beyond autonomous driving systems, such as medical image analysis, personalized recommendations, and more. In medical image analysis, automatic disease classification models must be continually updated to adapt to emerging diseases. Similarly, personalized recommendation systems require continual model updates to accommodate changing user preferences. Therefore, it is significant that our method can achieve better online continual learning in these fields as well.

CRediT authorship contribution statement

Ziqian Zeng: Supervision. **Jianwei Wang:** Writing – review & editing, Writing – original draft, Conceptualization. **Lin Wu:** Software. **Weikai Lu:** Supervision. **Huiping Zhuang:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No. 62406114, Grant No. 62306117), the Guangzhou Basic and Applied Basic Research Foundation, China (Grant No. 2023A04J1687, Grant No. 2024A04J3681), the South China University of Technology, China-TCL Technology Innovation Fund, the Fundamental Research Funds for the Central Universities, China (Grant No. 2024ZYGXZR074, 2023ZYGXZR023), Guangdong Basic and Applied Basic Research Foundation, China (Grant No. 2024A1515010220), CCF-Phytium Fund, China, and CAAI-MindSpore Open Fund, China, developed on OpenI Community.

References

[1] C. Badue, R. Guidolini, R.V. Carneiro, P. Azevedo, V.B. Cardoso, A. Forechi, L. Jesus, R. Berriel, T.M. Paixao, F. Mutz, et al., Self-driving cars: A survey, Expert Syst. Appl. 165 (2021) 113816.
[2] J. Janai, F. Güney, A. Behl, A. Geiger, et al., Computer vision for autonomous vehicles: Problems, datasets and state of the art, Found. Trends Comput. Graph. Vis. 12 (1–3) (2020) 1–308.
[3] L. Chen, Y. Li, C. Huang, B. Li, Y. Xing, D. Tian, L. Li, Z. Hu, X. Na, Z. Li, et al., Milestones in autonomous driving and intelligent vehicles: Survey of surveys, IEEE Trans. Intell. Veh. 8 (2022) 1046–1056.
[4] B. Fei, W. Yang, W.-M. Chen, Z. Li, Y. Li, T. Ma, X. Hu, L. Ma, Comprehensive review of deep learning-based 3D point cloud completion processing and analysis, IEEE Trans. Intell. Transp. Syst. 23 (2022) 22862–22883.
[5] Y. Guo, H. Wang, Q. Hu, H. Liu, L. Liu, M. Bennamoun, Deep learning for 3D point clouds: A survey, IEEE Trans. Pattern Anal. Mach. Intell. 43 (2020) 4338–4364.
[6] Z. Sun, Q. Ke, H. Rahmani, M. Bennamoun, G. Wang, J. Liu, Human action recognition from various data modalities: A review, IEEE Trans. Pattern Anal. Mach. Intell. 45 (2022) 3200–3225.
[7] H. Yu, Y. Luo, M. Shu, Y. Huo, Z. Yang, Y. Shi, Z. Guo, H. Li, X. Hu, J. Yuan, et al., Dair-v2x: A large-scale dataset for vehicle-infrastructure cooperative 3D object detection, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 21361–21370.
[8] H. Yu, W. Yang, H. Ruan, Z. Yang, Y. Tang, X. Gao, X. Hao, Y. Shi, Y. Pan, N. Sun, et al., V2x-seq: A large-scale sequential dataset for vehicle-infrastructure cooperative perception and forecasting, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 5486–5495.
[9] Y. Li, L. Ma, Z. Zhong, F. Liu, M.A. Chapman, D. Cao, J. Li, Deep learning for lidar point clouds in autonomous driving: A review, IEEE Trans. Neural Netw. Learn. Syst. 32 (2020) 3412–3432.
[10] K. Shaheen, M.A. Hanif, O. Hasan, M. Shafique, Continual learning for real-world autonomous systems: Algorithms, challenges and frameworks, J. Intell. Robot. Syst. 105 (2022) 9.
[11] S.C. Hoi, D. Sahoo, J. Lu, P. Zhao, Online learning: A comprehensive survey, Neurocomputing 459 (2021) 249–289.
[12] L. Wang, X. Zhang, H. Su, J. Zhu, A comprehensive survey of continual learning: Theory, method and application, IEEE Trans. Pattern Anal. Mach. Intell. (2024).
[13] E. Belouadah, A. Popescu, I. Kanellos, A comprehensive study of class incremental learning algorithms for visual tasks, Neural Netw. 135 (2021) 38–54, http://dx.doi.org/10.1016/J.NEUNET.2020.12.003.
[14] H. Zhuang, Z. Weng, H. Wei, R. Xie, K.-A. Toh, Z. Lin, ACIL: Analytic class-incremental learning with absolute memorization and privacy protection, Adv. Neural Inf. Process. Syst. 35 (2022) 11602–11614.
[15] J.M. Johnson, T.M. Khoshgoftaar, Survey on deep learning with class imbalance, J. Big Data 6 (2019) 27, http://dx.doi.org/10.1186/S40537-019-0192-5.
[16] J. Dong, Y. Cong, G. Sun, B. Ma, L. Wang, I3dol: Incremental 3D object learning without catastrophic forgetting, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2021, pp. 6066–6074.
[17] M. Yazgan, T. Graf, M. Liu, J.M. Zoellner, A survey on intermediate fusion methods for collaborative perception categorized by real world challenges, 2024, arXiv preprint arXiv:2404.16139.
[18] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, J. Xiao, 3D ShapeNets: A deep representation for volumetric shapes, in: IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015, 2015, pp. 1912–1920, http://dx.doi.org/10.1109/CVPR.2015.7298801.
[19] A. Geiger, P. Lenz, R. Urtasun, Are we ready for autonomous driving? the kitti vision benchmark suite, in: 2012 IEEE Conference on Computer Vision and Pattern Recognition, IEEE, 2012, pp. 3354–3361.
[20] G.J. Brostow, J. Fauqueur, R. Cipolla, Semantic object classes in video: A high-definition ground truth database, Pattern Recognit. 30 (2) (2009) 88–97, http://dx.doi.org/10.1016/J.PATREC.2008.04.005.
[21] E. Alberti, A. Tavera, C. Masone, B. Caputo, IDDA: A large-scale multi-domain dataset for autonomous driving, IEEE Robotics Autom. Lett. 5 (4) (2020) 5526–5533, http://dx.doi.org/10.1109/LRA.2020.3009075.
[22] S.R. Richter, V. Vineet, S. Roth, V. Koltun, Playing for data: Ground truth from computer games, in: Computer Vision - ECCV 2016 - 14th European Conference, Amsterdam, the Netherlands, October 11-14, 2016, Proceedings, Part II, 2016, pp. 102–118, http://dx.doi.org/10.1007/978-3-319-46475-6_7.

[23] R. Xu, Y. Guo, X. Han, X. Xia, H. Xiang, J. Ma, OpenCDA: an open cooperative driving automation framework integrated with co-simulation, in: 2021 IEEE International Intelligent Transportation Systems Conference, ITSC, IEEE, 2021, pp. 1155–1162.

[24] D. Krajzewicz, J. Erdmann, M. Behrisch, L. Bieker, Recent development and applications of SUMO-simulation of urban mobility, Int. J. Adv. Syst. Meas. 5 (3&4) (2012).

[25] M. Alibeigi, W. Ljungbergh, A. Tonderski, G. Hess, A. Lilja, C. Lindström, D. Motorniuk, J. Fu, J. Widahl, C. Petersson, Zenseact open dataset: A large-scale and diverse multimodal dataset for autonomous driving, in: IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023, 2023, pp. 20121–20131, http://dx.doi.org/10.1109/ICCV51070.2023.01846.

[26] P. Sun, H. Kretzschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine, V. Vasudevan, W. Han, J. Ngiam, H. Zhao, A. Timofeev, S. Ettinger, M. Krivokon, A. Gao, A. Joshi, Y. Zhang, J. Shlens, Z. Chen, D. Anguelov, Scalability in perception for autonomous driving: Waymo open dataset, in: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020, 2020, pp. 2443–2451, http://dx.doi.org/10.1109/CVPR42600.2020.00252, URL https://openaccess.thecvf.com/content_CVPR_2020/html/Sun_Scalability_in_Perception_for_Autonomous_Driving_Waymo_Open_Dataset_CVPR_2020_paper.html.

[27] J. Mao, M. Niu, C. Jiang, H. Liang, J. Chen, X. Liang, Y. Li, C. Ye, W. Zhang, Z. Li, J. Yu, C. Xu, H. Xu, One million scenes for autonomous driving: ONCE dataset, in: Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, Virtual, 2021, URL https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/67c6a1e7ce56d3d6fa748ab6d9af3fd7-Abstract-round1.html.

[28] Y. Li, D. Ma, Z. An, Z. Wang, Y. Zhong, S. Chen, C. Feng, V2X-sim: Multi-agent collaborative perception dataset and benchmark for autonomous driving, IEEE Robot. Autom. Lett. 7 (4) (2022) 10914–10921.

[29] H. Zhang, C. Wang, S. Tian, B. Lu, L. Zhang, X. Ning, X. Bai, Deep learning-based 3D point cloud classification: A systematic survey and outlook, Displays (2023) 102456.

[30] E. Grilli, F. Menna, F. Remondino, A review of point clouds segmentation and classification algorithms, Int. Arch. Photogr. Remote Sens. Spat. Inf. Sci. 42 (2017) 339–344.

[31] C.R. Qi, H. Su, K. Mo, L.J. Guibas, Pointnet: Deep learning on point sets for 3D classification and segmentation, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 652–660.

[32] C.R. Qi, L. Yi, H. Su, L.J. Guibas, Pointnet++: Deep hierarchical feature learning on point sets in a metric space, Adv. Neural Inf. Process. Syst. 30 (2017).

[33] L. Zhao, Z. Zeng, Dap-SiMT: divergence-based adaptive policy for simultaneous machine translation, Int. J. Mach. Learn. Cybern. (2024) 1–20.

[34] Z. Li, D. Hoiem, Learning without forgetting, IEEE Trans. Pattern Anal. Mach. Intell. 40 (2017) 2935–2947.

[35] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A.A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, et al., Overcoming catastrophic forgetting in neural networks, Proc. Natl. Acad. Sci. 114 (2017) 3521–3526.

[36] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, C.H. Lampert, Icarl: Incremental classifier and representation learning, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 2001–2010.

[37] Y. Liu, Y. Su, A.-A. Liu, B. Schiele, Q. Sun, Mnemonics training: Multi-class incremental learning without forgetting, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 12245–12254.

[38] T. Lesort, H. Caselles-Dupré, M. Garcia-Ortiz, A. Stoian, D. Filliat, Generative models from the perspective of continual learning, in: 2019 International Joint Conference on Neural Networks, IJCNN, 2019, pp. 1–8.

[39] H. Shin, J.K. Lee, J. Kim, J. Kim, Continual learning with deep generative replay, Adv. Neural Inf. Process. Syst. 30 (2017).

[40] P. Guo, M.R. Lyu, A pseudoinverse learning algorithm for feedforward neural networks with stacked generalization applications to software reliability growth data, Neurocomputing 56 (2004) 101–121.

[41] Y. Eldar, M. Lindenbaum, M. Porat, Y.Y. Zeevi, The farthest point strategy for progressive image sampling, IEEE Trans. Image Process. 6 (1997) 1305–1315.

[42] M. Han, L. Wang, L. Xiao, H. Zhang, C. Zhang, X. Xu, J. Zhu, QuickFPS: Architecture and algorithm co-design for farthest point sampling in large-scale point clouds, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. (2023).

[43] M. Abadi, A. Chu, I. Goodfellow, H.B. McMahan, I. Mironov, K. Talwar, L. Zhang, Deep learning with differential privacy, in: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, 2016, pp. 308–318.

[44] A.D. Amirruddin, F.M. Muharam, M.H. Ismail, N.P. Tan, M.F. Ismail, Synthetic Minority Over-sampling TEchnique (SMOTE) and Logistic Model Tree (LMT)-Adaptive Boosting algorithms for classifying imbalanced datasets of nutrient and chlorophyll sufficiency levels of oil palm (*Elaeis guineensis*) using spectroradiometers and unmanned aerial vehicles, Comput. Electron. Agric. 193 (2022) 106646, http://dx.doi.org/10.1016/J.COMPAG.2021.106646.

[45] E. Verwimp, K. Yang, S. Parisot, L. Hong, S. McDonagh, E. Pérez-Pellitero, M. De Lange, T. Tuytelaars, Clad: A realistic continual learning benchmark for autonomous driving, Neural Netw. 161 (2023) 659–669.